

Applied Machine Learning Project 1

Presented by Group 8

Consist of – Asma Akter, Sazzad Hossain, Md. Abdul Wahid Raju,
Shusmita Dutta

Introduction

Customer Churn is a critical issue for telecommunication companies, leading to significant revenue loss. This project aims to build predictive models to forecast customer churn and to gain interpretable insights into the factors influencing churn.

Objective of EDA

The purpose of EDA is to:

- Analyze dataset structure: distributions, outliers, skewness
 - Examine correlations among numerical features
 - Explore categorical feature cardinality
 - Study churn vs. non-churn population imbalance
- Identify missing values and propose imputation strategies

Dataset Overview

The dataset loaded using pandas.

The dataset contains 7043 observation with 21 features.

The target variable is churn.

The data has various types including numerical features, categorical features and binary features.

Statistical Summary (df.describe(include="all"))

This function provides a comprehensive summary for both numerical and categorical variables. For numerical values, it shows mean, standard deviation, min, max and quartiles. That numerical summaries help detect outliers and skewed distributions. For categorical values it shows Count, Unique, Top and Frequency.

Conversion Of Totalcharges to numeric

Totalcharges is stored as an object due to blank value. It is converted to Numeric. Valid numbers are converted correctly. Blank values are converted to NaN.

```
#change total charge from string to numeric(#Convert TotalCharges to numeric (blank → NaN))
df_clean["TotalCharges"] = pd.to_numeric(
    df_clean["TotalCharges"], errors="coerce"
)
```

Missing value analysis

After conversion, all columns except TotalCharges has zero missing values while Total charges has 11 missing values.

	tenure	MonthlyCharges	TotalCharges
488	0	52.55	NaN
753	0	20.25	NaN
936	0	80.85	NaN
1082	0	25.75	NaN
1340	0	56.05	NaN
3331	0	19.85	NaN
3826	0	25.35	NaN
4380	0	20.00	NaN
5218	0	19.70	NaN
6670	0	73.35	NaN
6754	0	61.90	NaN

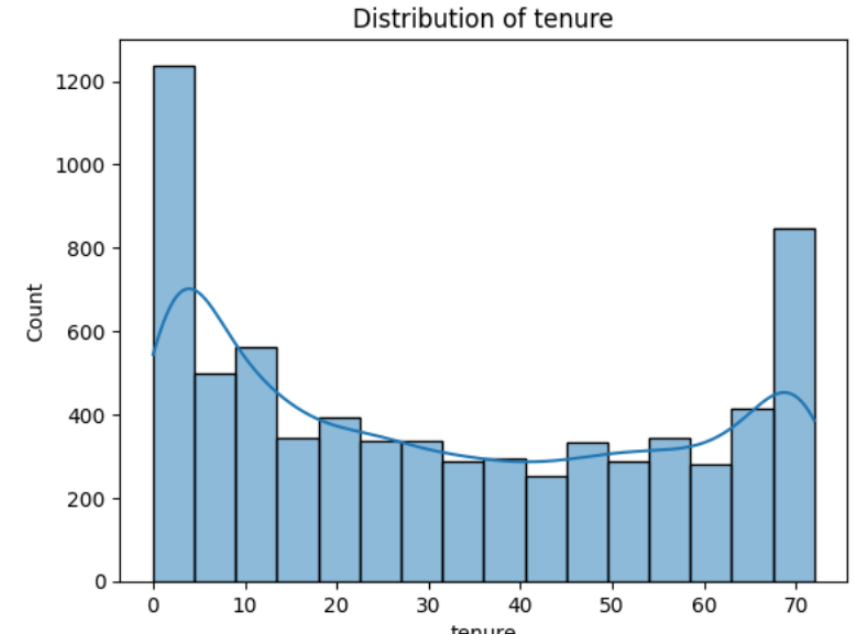
Missing Value Imputation

```
#Imputation: If tenure == 0, set TotalCharges = 0  
df_clean["TotalCharges"] = df_clean["TotalCharges"].fillna(0)  
  
df_clean["TotalCharges"].isna().sum()
```

After imputation there is no missing values.

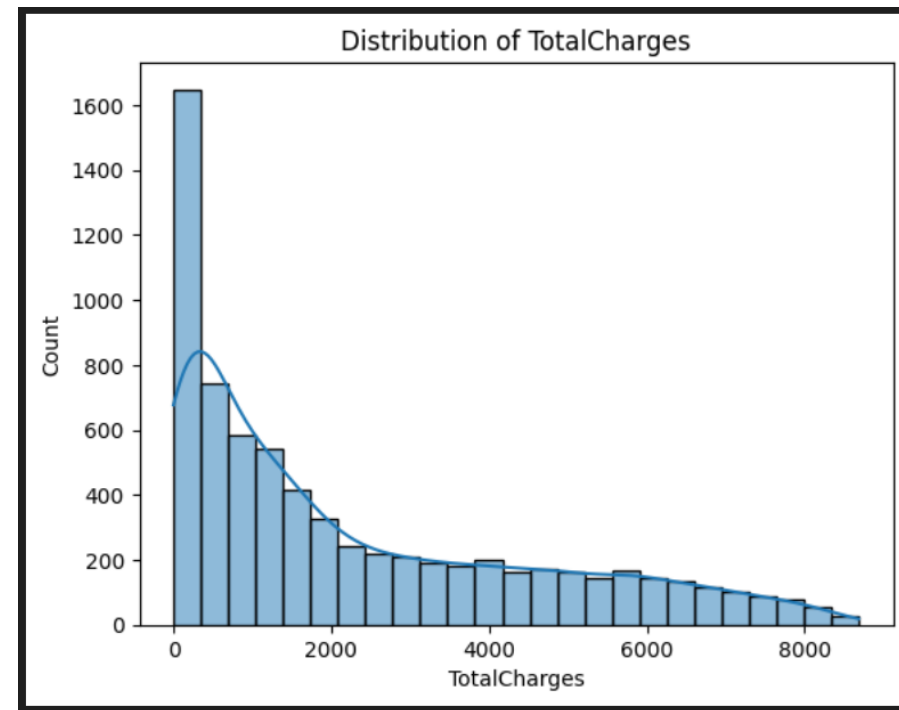
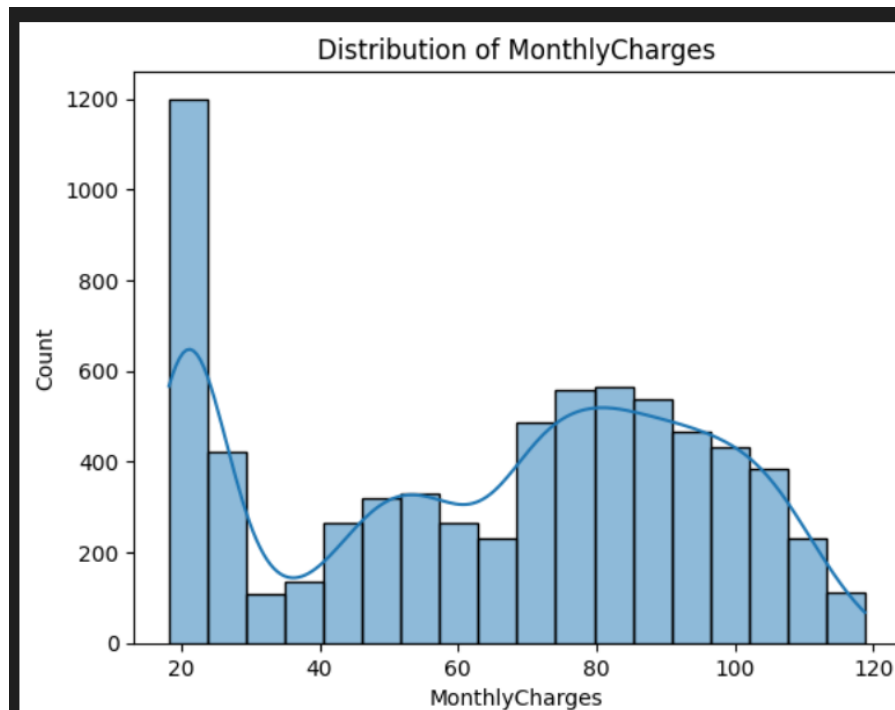
Distribution Analysis

-
- Distribution analysis revealed that *tenure* is right-skewed,
 - with a large proportion of customers in their initial months and another concentration near the maximum contract duration.



Distribution Analysis

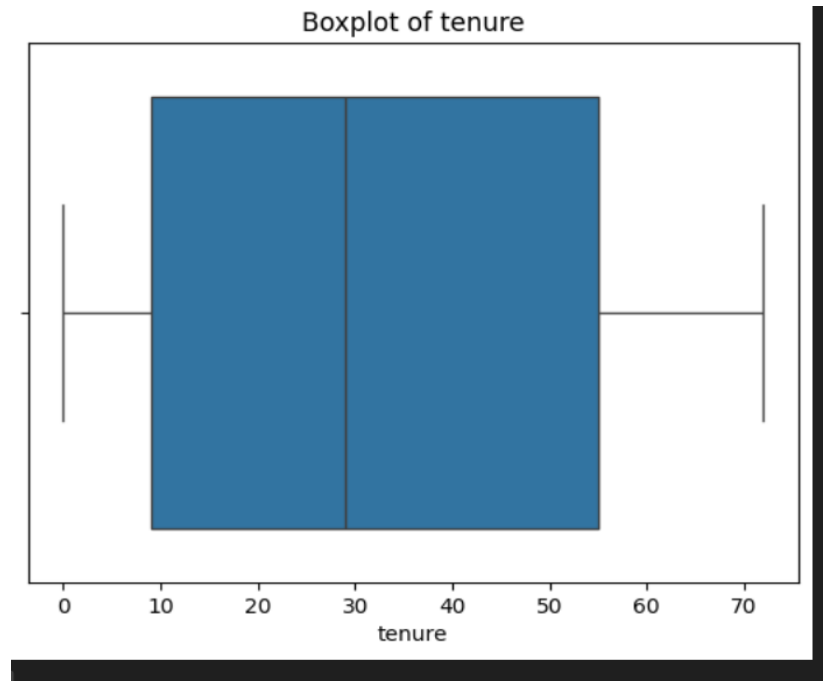
- while MonthlyCharges exhibited slight bimodality and
- TotalCharges was highly right-skewed and strongly correlated with tenure.



Outlier Analysis

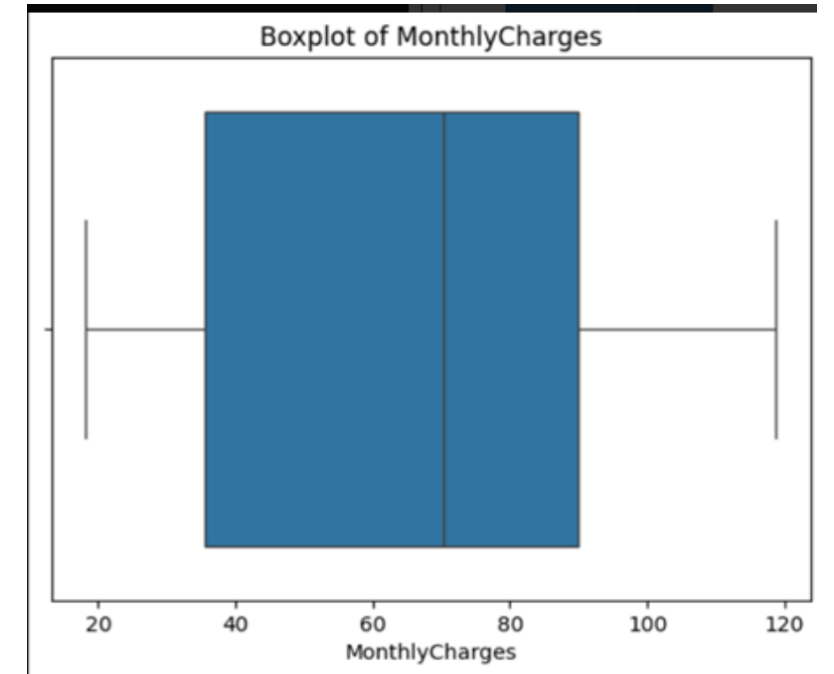
Boxplot of tenure shows :

- Tenure ranges from very short- term to long-term customers, indicating diverse customer base.
- The median lies near the center shows a fairly balanced distribution.



Boxplot of monthlycharges shows :

The median lies near the center that shows balanced distribution.



Distribution By Churn Status

Tenure distribution by churn:

- Churned customers are heavily concentrated at low tenure values, especially early months.
- Non-churned customers are more frequent at higher tenure levels mans long term relationship.

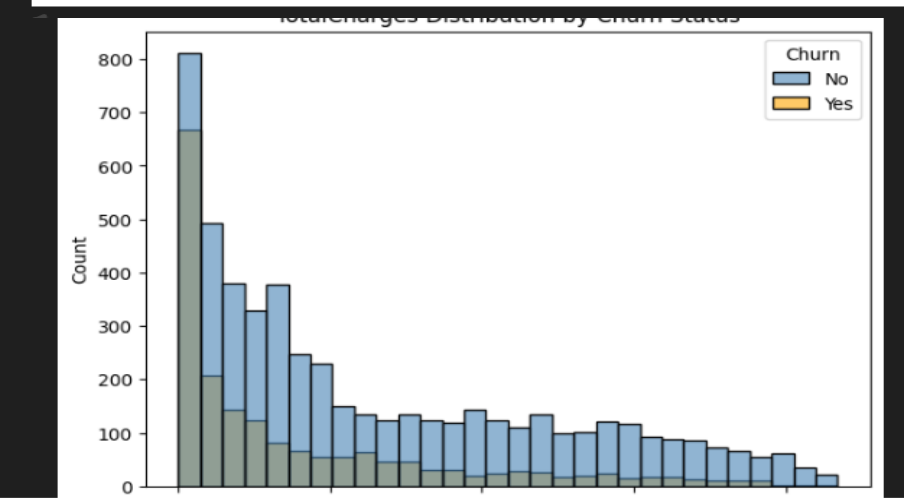
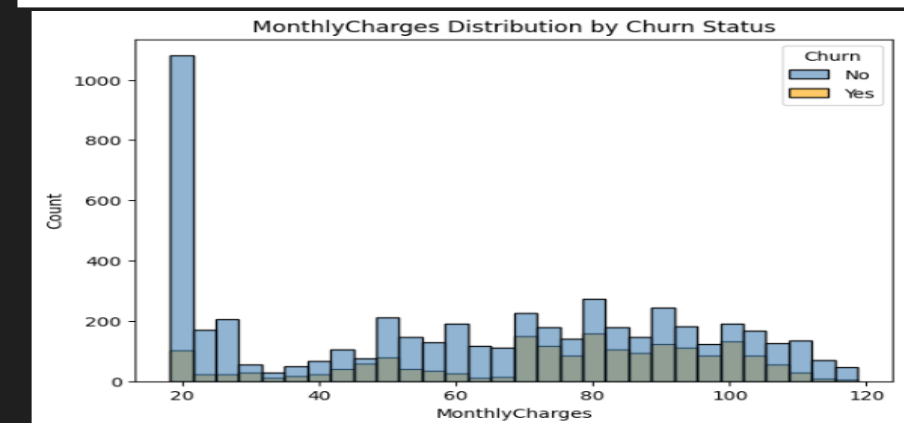
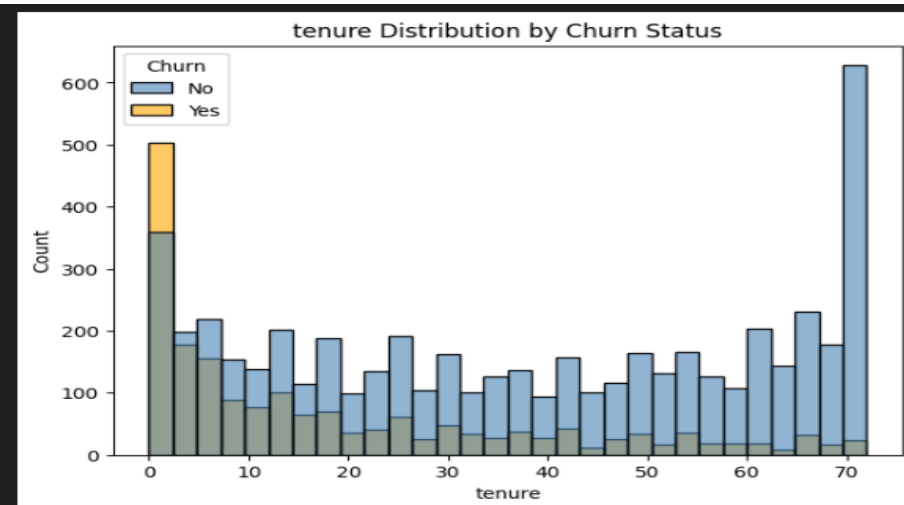
Monthlycharges distribution by churn:

- Customers with higher monthlycharges are more likely to churn.
- Customers with lower monthlycharges tend to stay longer.

Totalcharges distributionby churn status:

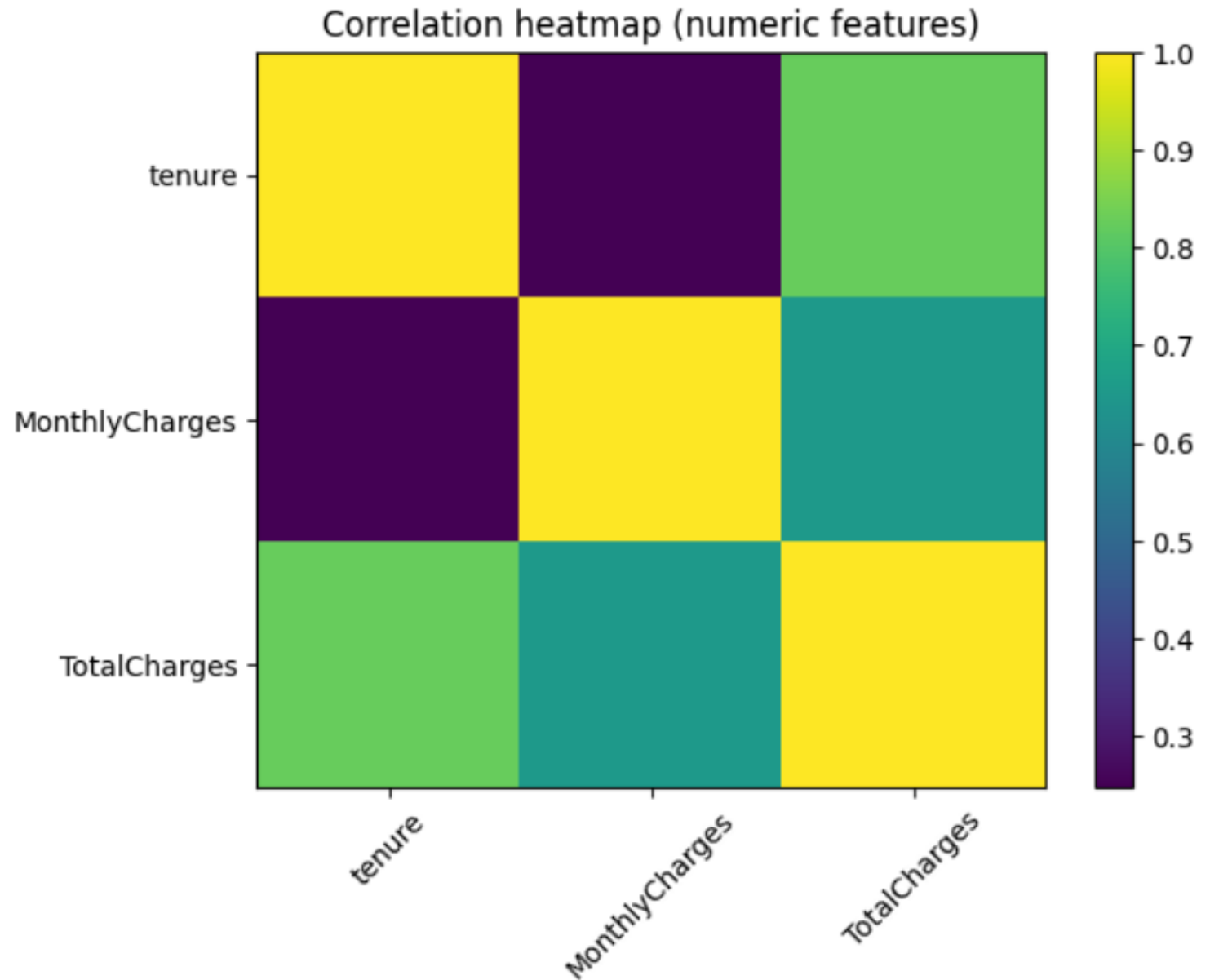
Customers with low total charges are more likely to churn.

Customer with high total charges usually stays with company.



Correlation Heatmap

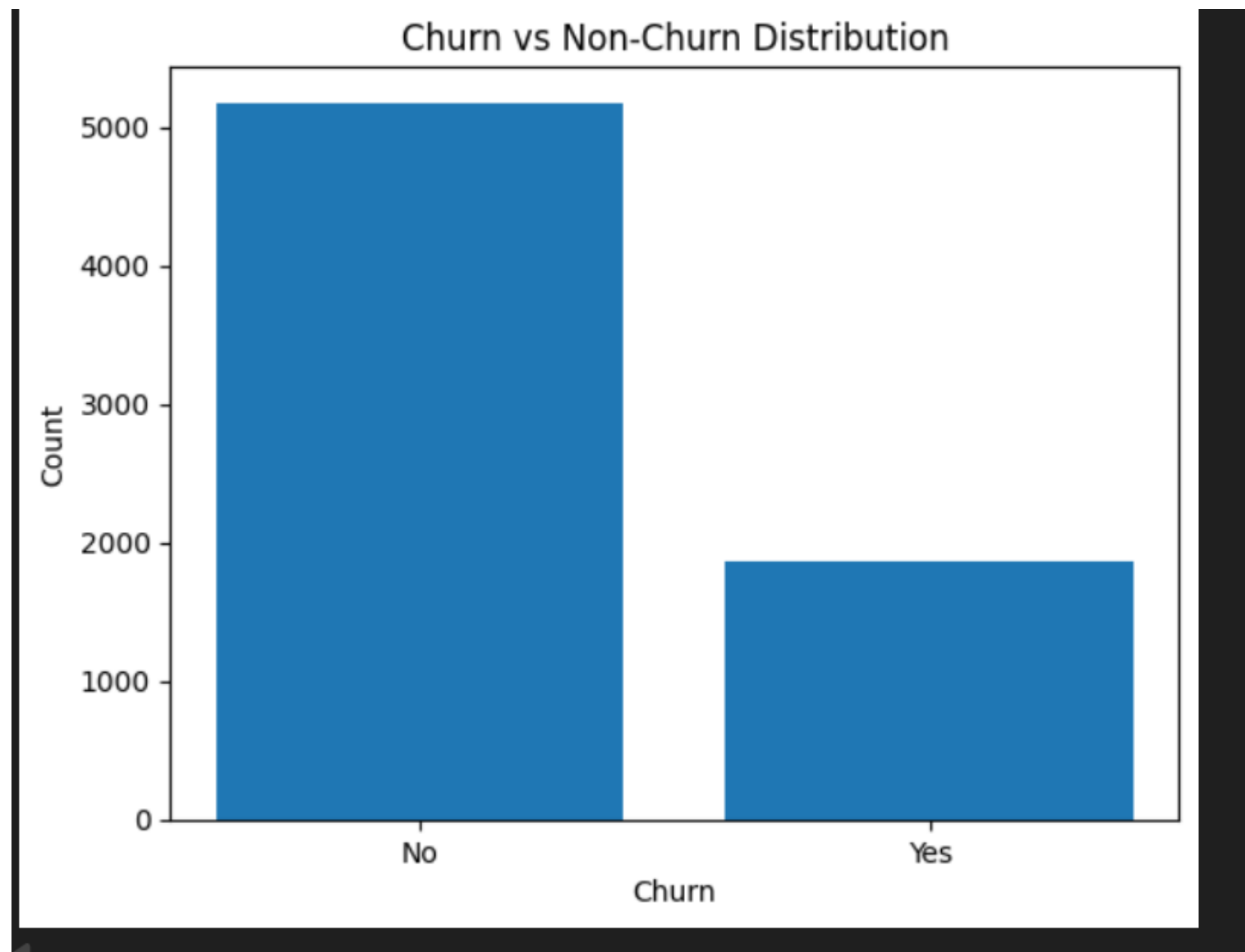
Correlation analysis showed a strong positive relationship between *tenure* and *TotalCharges* (0.83) and a moderate correlation between *MonthlyCharges* and *TotalCharges* (0.65)



Churn vs Non-Churn Distribution

This plot shows a strong imbalance between churn and non-churn customers.

Most customers do not churn and the percentage is 73.463013 while churn percentage is 26.536987.



+

•

○

Task 2 – Shallow Learning Approaches

- **Content**
- Dataset: Telco Customer Churn (Kaggle)
- Goal: Predict customer churn (Yes / No)

Data Preprocessing

```
#define X and y and manually put 0 and 1 in target variable
X = df1.drop(columns=["Churn"])
y = df1["Churn"].map({"No": 0, "Yes": 1}) # 0 = No churn, 1 = churn

y.value_counts(normalize=True)
```

```
...      proportion
Churn
0      0.73463
1      0.26537
```

dtype: float64

```
#Train/Test split (STRATIFIED) + preprocessing
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
#Cell 4 :column groups
num_features = ["tenure", "MonthlyCharges", "TotalCharges"]
cat_features = [c for c in X.columns if c not in num_features]
```

- Removed non-informative identifier (customerID)
- Target encoding: Churn (No → 0, Yes → 1)
- Categorical features → One-Hot Encoding
- Numerical features → Standardization (model-dependent)
- Stratified 80–20 train–test split

```

# Model 1: Logistic Regression (ensemble-compatible: keep TotalCharges)
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, GridSearchCV

# Use the same feature set as RF/GB (do NOT drop TotalCharges)
X_train_lr = X_train.copy()
X_test_lr = X_test.copy()

num_lr = ["tenure", "MonthlyCharges", "TotalCharges"]
cat_lr = [c for c in X_train_lr.columns if c not in num_lr]

preprocess_lr = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_lr),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_lr)
    ]
)

lr = LogisticRegression(max_iter=2000, class_weight="balanced", solver="liblinear")

pipe_lr = Pipeline(steps=[
    ("prep", preprocess_lr),
    ("model", lr)
])

param_grid_lr = {
    "model__C": [0.01, 0.1, 1, 10]
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid_lr = GridSearchCV(
    pipe_lr, param_grid_lr, cv=cv, scoring="f1", n_jobs=-1
)

grid_lr.fit(X_train_lr, y_train)

grid_lr.best_params_, grid_lr.best_score_

```

Logistic Regression (Baseline Model)

- Used as baseline binary classifier
- Preprocessing:
 - One-Hot Encoding (categorical features)
 - Standardization (numerical features)
- Class imbalance handled using class_weight = "balanced"
- Hyperparameters tuned with GridSearchCV (F1-score)
- **Test Performance**
 - Accuracy $\approx$ **0.74**
 - F1-score (Churn) $\approx$ **0.61**

Random Forest (Tree-Based Ensemble)

```
[8]: #Random Forest pipeline + tuning
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    random_state=42,
    class_weight="balanced"
)

pipe_rf = Pipeline(steps=[
    ("prep", ColumnTransformer(
        transformers=[
            ("num", "passthrough", num_features),
            ("cat", OneHotEncoder(handle_unknown="ignore"), cat_features)
        ]
    )),
    ("model", rf)
])

param_grid_rf = {
    "model__n_estimators": [200, 400],
    "model__max_depth": [None, 10, 20],
    "model__min_samples_split": [2, 10],
    "model__min_samples_leaf": [1, 5]
}

grid_rf = GridSearchCV(
    pipe_rf,
    param_grid_rf,
    cv=cv,
    scoring="f1",
    n_jobs=-1
)

grid_rf.fit(X_train, y_train)

grid_rf.best_params_, grid_rf.best_score_
```

- **Slide Content**
- Tree-based ensemble model (bagging)
- Modeling priorities:
 - Capture non-linear feature interactions
 - Reduce overfitting via ensemble averaging
- Preprocessing decisions:
 - One-Hot Encoding for categorical features
 - No numerical feature scaling (scale-invariant)
- Class imbalance handled using `class_weight = "balanced"`
- Hyperparameters tuned using GridSearchCV (F1-score)
- **Test Performance**
- Accuracy $\approx$ **0.77**
- F1-score (Churn) $\approx$ **0.62**

```

0]: #Cell - Gradient Boosting tuning
from sklearn.ensemble import GradientBoostingClassifier

gb = GradientBoostingClassifier(random_state=42)

pipe_gb = Pipeline(steps=[
    ("prep", ColumnTransformer(
        transformers=[
            ("num", "passthrough", num_features),
            ("cat", OneHotEncoder(handle_unknown="ignore"), cat_features)
        ]
    )),
    ("model", gb)
])

param_grid_gb = {
    "model__n_estimators": [100, 200],
    "model__learning_rate": [0.05, 0.1],
    "model__max_depth": [3, 5]
}

grid_gb = GridSearchCV(
    pipe_gb,
    param_grid_gb,
    cv=cv,
    scoring="f1",
    n_jobs=-1
)

grid_gb.fit(X_train, y_train)

grid_gb.best_params_, grid_gb.best_score_

```

Gradient Boosting: Modeling Strategy

- Sequential ensemble of decision trees (boosting)
- Modeling priorities:
 - Reduce bias by correcting previous errors
 - Improve predictive accuracy on complex patterns
- Preprocessing decisions:
 - One-Hot Encoding for categorical features
 - No numerical feature scaling required
- Hyperparameters tuned:
 - Learning rate
 - Number of estimators
 - Tree depth
- Model optimized using GridSearchCV (F1-score)
- **Test Performance**
- Accuracy $\approx$ **0.80**
- F1-score (Churn) $\approx$ **0.58**

Voting Ensemble (Soft Voting)

```
#Voting Ensemble(Soft)
best_lr = grid_lr.best_estimator_
best_rf = grid_rf.best_estimator_
best_gb = grid_gb.best_estimator_
```

```
#create the voting classifier
from sklearn.ensemble import VotingClassifier

voting = VotingClassifier(
    estimators=[
        ("lr", best_lr),
        ("rf", best_rf),
        ("gb", best_gb)
    ],
    voting="soft" # use probabilities
)
```

```
#train voting
voting.fit(X_train, y_train)
```

- Models combined:
 - Logistic Regression
 - Random Forest
 - Gradient Boosting
- Soft voting:
 - Averages predicted probabilities
 - Suitable for imbalanced data
- Selected based on F1-score
- **Performance**
- Accuracy $\approx$ **0.78**
- F1-score (Churn) $\approx$ **0.63** (*Best*)

Model Comparison & Selection

- Although Gradient Boosting had the highest accuracy, the voting ensemble achieved the best F1-score, which is more important for churn prediction.

Model	Accuracy	F1 (Churn)
Logistic Regression	~0.74	~0.61
Random Forest	~0.77	~0.62
Gradient Boosting	~0.80	~0.58
Voting Ensemble	~0.78	~0.63
Final Model: Soft Voting Ensemble		



Neural Network & Autoencoder Approach

- Feedforward Network (FFNN)
- Hyperparameter tuning
- Autoencoder + Logistic Regression

NOW THAT THE DATASET PREPROCESSING AND SHALLOW MACHINE LEARNING MODELS ARE COVERED, I'LL PRESENT THE NEURAL NETWORK-BASED APPROACH WE USED, INCLUDING A FEEDFORWARD NEURAL NETWORK AND AN AUTOENCODER-BASED METHOD.

Preprocessing for Neural Networks

Data Preparation for FFNN

```
#Train / Validation / Test split (with stratification)
from sklearn.model_selection import train_test_split

# 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=42, stratify=y
)

# From train: 80% train, 20% val -> overall: 64% train, 16% val, 20% test
X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42, stratify=y_train
)
```

- ▶ Neural networks require more careful preprocessing compared to tree-based models.
- ▶ First, we encoded the target variable churn as 0 for non-churn and 1 for churn. All categorical features were converted using one-hot encoding so that the network could process them numerically.
- ▶ We then split the data into training, validation, and test sets using stratified sampling. This ensures that the proportion of churned and non-churned customers remains consistent across all splits.
- ▶ Finally, we standardized all features using StandardScaler, which is very important for neural networks because it stabilizes gradient updates and speeds up convergence."

Feedforward Neural Network (FFNN)

Regularized FFNN Architecture

- ▶ Binary classification using sigmoid output
- ▶ Two hidden layers (64, 32 neurons)
- ▶ Regularization to prevent overfitting
- ▶ Adam optimizer + binary cross-entropy loss

```
#Define a SIMPLE FFNN model (baseline)
model = keras.Sequential([
    layers.Input(shape=(X_train_s.shape[1],)), # number of input features = 23
    layers.Dense(64, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(1, activation="sigmoid") # binary classification
])
```

This slide shows the core architecture of our feedforward neural network.

We used two hidden layers with 64 and 32 neurons, which allows the model to capture non-linear relationships between customer behavior and churn. The output layer uses a sigmoid activation function, so the model outputs a churn probability between 0 and 1.

To prevent overfitting, we applied multiple regularization techniques. L2 regularization penalizes large weights, batch normalization stabilizes training and improves convergence, and dropout randomly deactivates neurons during training to improve generalization.

Overall, this architecture balances model complexity and robustness.

Model Tuning & Training Strategy

Training & Hyperparameter Experiments

```
#Early Stopping callback
early_stop = keras.callbacks.EarlyStopping(
    ... monitor="val_loss",
    ... patience=10,
    ... restore_best_weights=True
)
```

After defining the architecture, we experimented with different training configurations. We tested multiple optimizers, learning rates, and activation functions such as ReLU and Tanh.

Since customer churn is an imbalanced problem, accuracy alone is not sufficient. Therefore, we selected the best model based on validation F1-score, which balances precision and recall.

We also used early stopping based on validation loss. This stops training automatically when the model stops improving and restores the best-performing weights, helping us avoid overfitting.

Test Results (FFNN)

FFNN Performance on Test Set

- ▶ Using the best-performing configuration, we evaluated the FFNN on the unseen test set. The model achieved around 80% accuracy and an F1-score of approximately 0.60.
- ▶ While the accuracy is relatively high, the F1-score gives a more realistic picture of performance in this imbalanced setting. We also analyzed the confusion matrix to understand where the model makes false positives and false negatives, which is important for business decision-making.

Autoencoder-Based Approach

Autoencoder for Feature Compression

- ▶ In addition to the FFNN, we explored an autoencoder-based approach for feature learning.
- ▶ An autoencoder is an unsupervised neural network that learns to compress the input data into a lower-dimensional representation and then reconstruct it. In our case, we compressed the original 23 features into 8 latent features.
- ▶ After training the autoencoder using reconstruction loss, we extracted the latent features and used them to train a logistic regression classifier for churn prediction.

FFNN vs Autoencoder Comparison

Model Comparison

- ▶ This table compares the raw FFNN with the autoencoder-based approach.
- ▶ The FFNN achieved slightly better accuracy and F1-score compared to the autoencoder combined with logistic regression. This suggests that compressing the features into a lower-dimensional space may have removed some discriminative information that is useful for churn prediction.
- ▶ As a result, the direct FFNN trained on the full feature set performed more effectively.

Summary

- ▶ To summarize, the feedforward neural network was able to capture complex, non-linear patterns in customer churn data and achieved solid predictive performance. The autoencoder-based approach, while interesting, was slightly less effective in this case.
- ▶ Overall, we selected models based on F1-score to handle class imbalance properly. Next, Asma Akter will discuss the overall model comparison, interpretability using SHAP values, and the business insights derived from our results.

TASK 4: MODEL COMPARISON & INTERPRETABILITY

By Asma Akter

OVERVIEW:

- Compare different models' performance in accordance with the accuracy and F1 score. Error analysis of misclassified users Use
- SHAP to determine important features and model differences.
- Conclude with deployment decision, most churn features, and the
- possible ways to improve the performance.

MODELS COMPARED

Shallow models: Logistic Regression(tuned), Random Forest (tuned), and Gradient Boosting (tuned).

- Soft Voting Ensemble.
- Feed-Forward Neural Network (FFNN),
Autoencoder .

MODEL PERFORMANCE RESULTS

	model	accuracy	f1
3	Voting Ensemble (soft)	0.780696	0.626360
1	Random Forest (tuned)	0.770050	0.624130
0	Logistic Regression (tuned)	0.739532	0.612460
4	Raw FFNN (23 features)	0.796300	0.600800
5	Autoencoder (8 features) + Logistic Regression	0.723200	0.596300
2	Gradient Boosting (tuned)	0.802697	0.581325

ERROR ANALYSIS: Voting Ensemble (Churn Prediction)

Number of False Positives: 194

Number of False Negatives: 115

- The model flags more customers as at risk than necessary.
- Mises fewer actual churners.
- Indicates a conservative prediction strategy.

WHY ERRORS HAPPEN:

- **Over-reliance on strong churn indicators:**

Month-to-month + Fiber optic + No support
= churn(But that does not happen always)

- **Conservative Voting Ensemble behaviour:**

The model prefers to warn early.
It sacrifices precision to improve recall

- **Missing behavioral features:**

The model includes(Services, Contracts,Charges) these features, but There are features such as customer satisfaction, Complaints, and Discounts. These missing features could explain why these customers stayed.

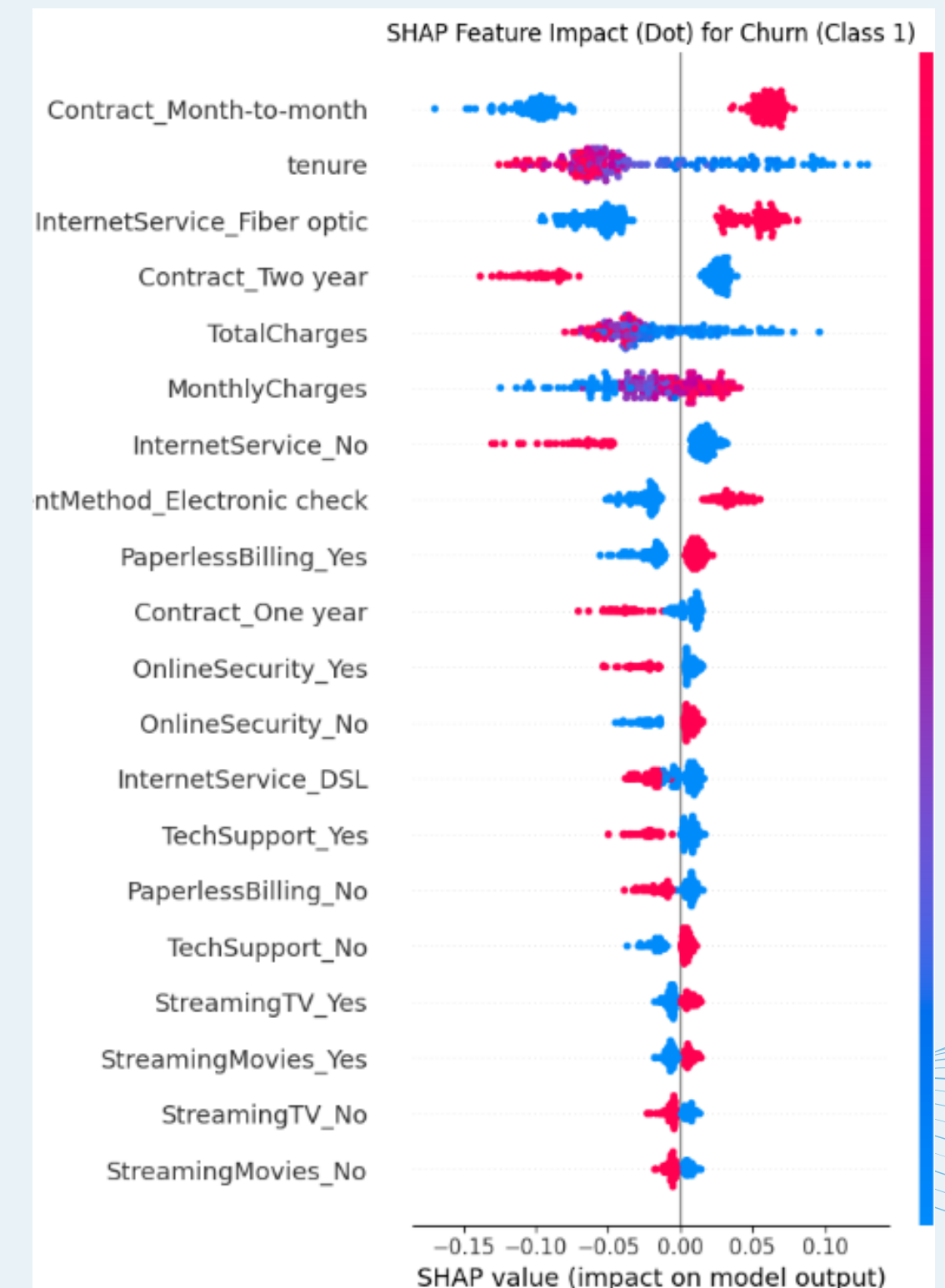
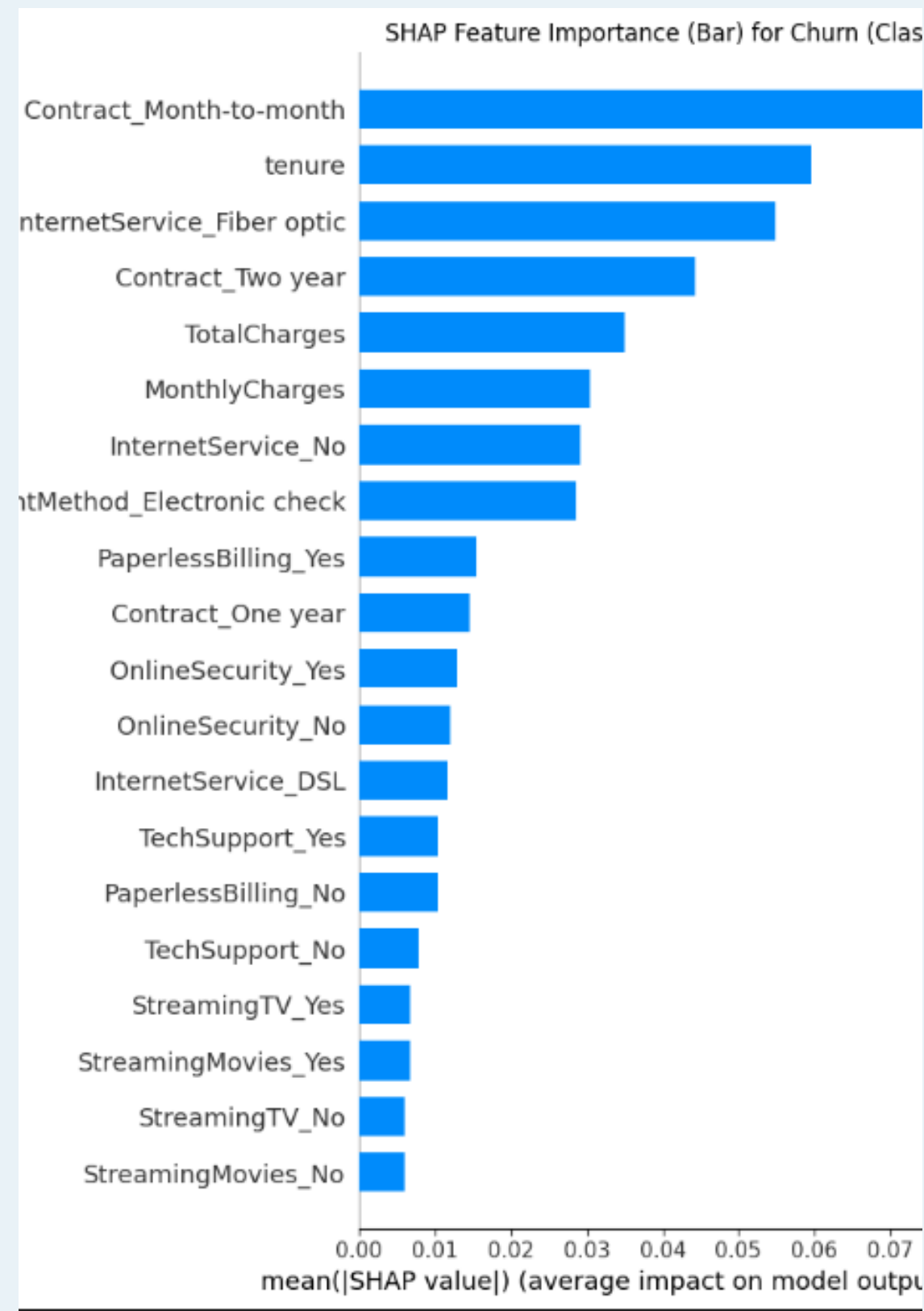
SHAP ANALYSIS: EXPLAINING MODEL PREDICTIONS

RANDOM FOREST

Random Forest explanation

```
shap.summary_plot(  
    shap_values_rf[:, :, 1],  
    X_test_shap_subset,  
    plot_type="bar"  
)
```

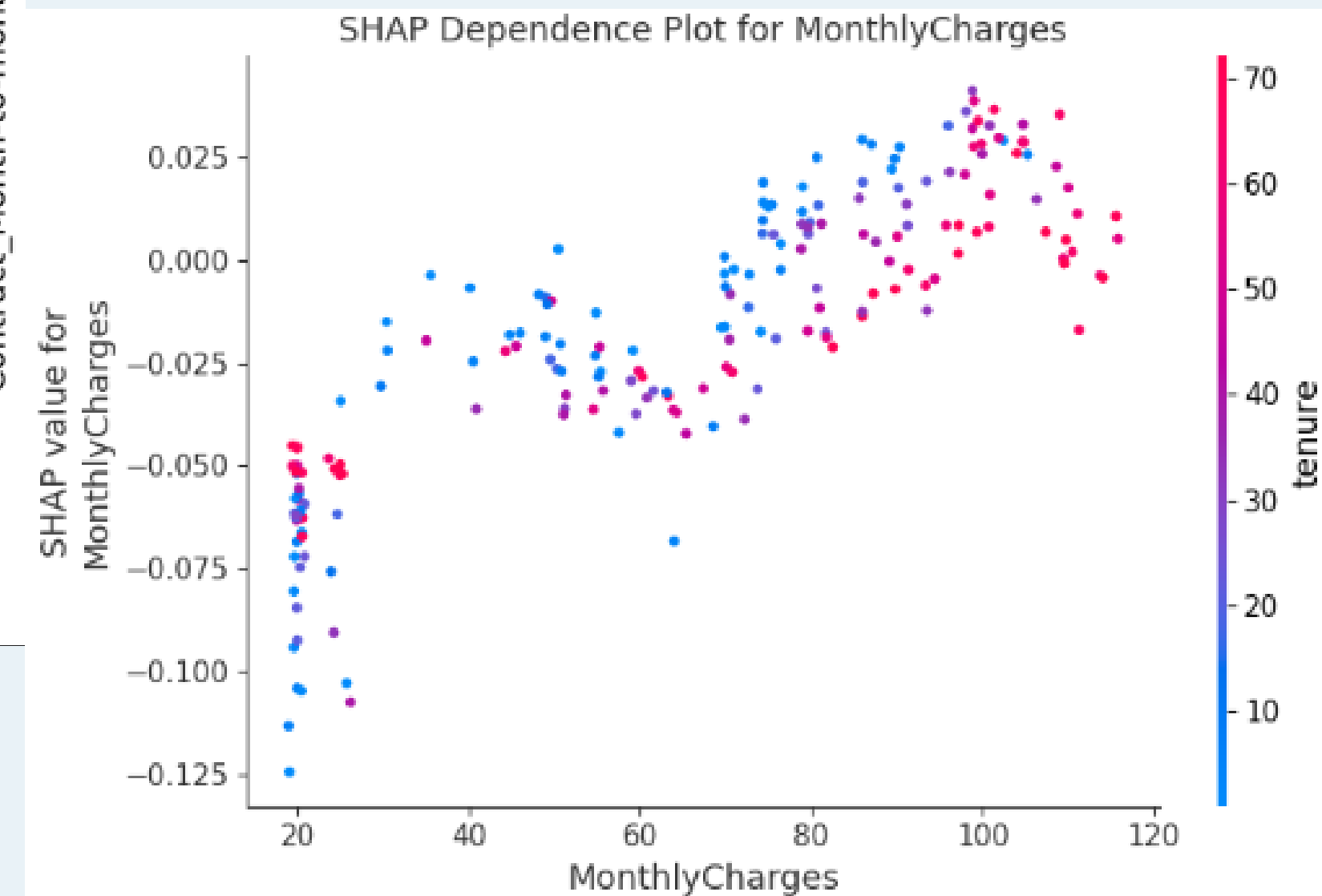
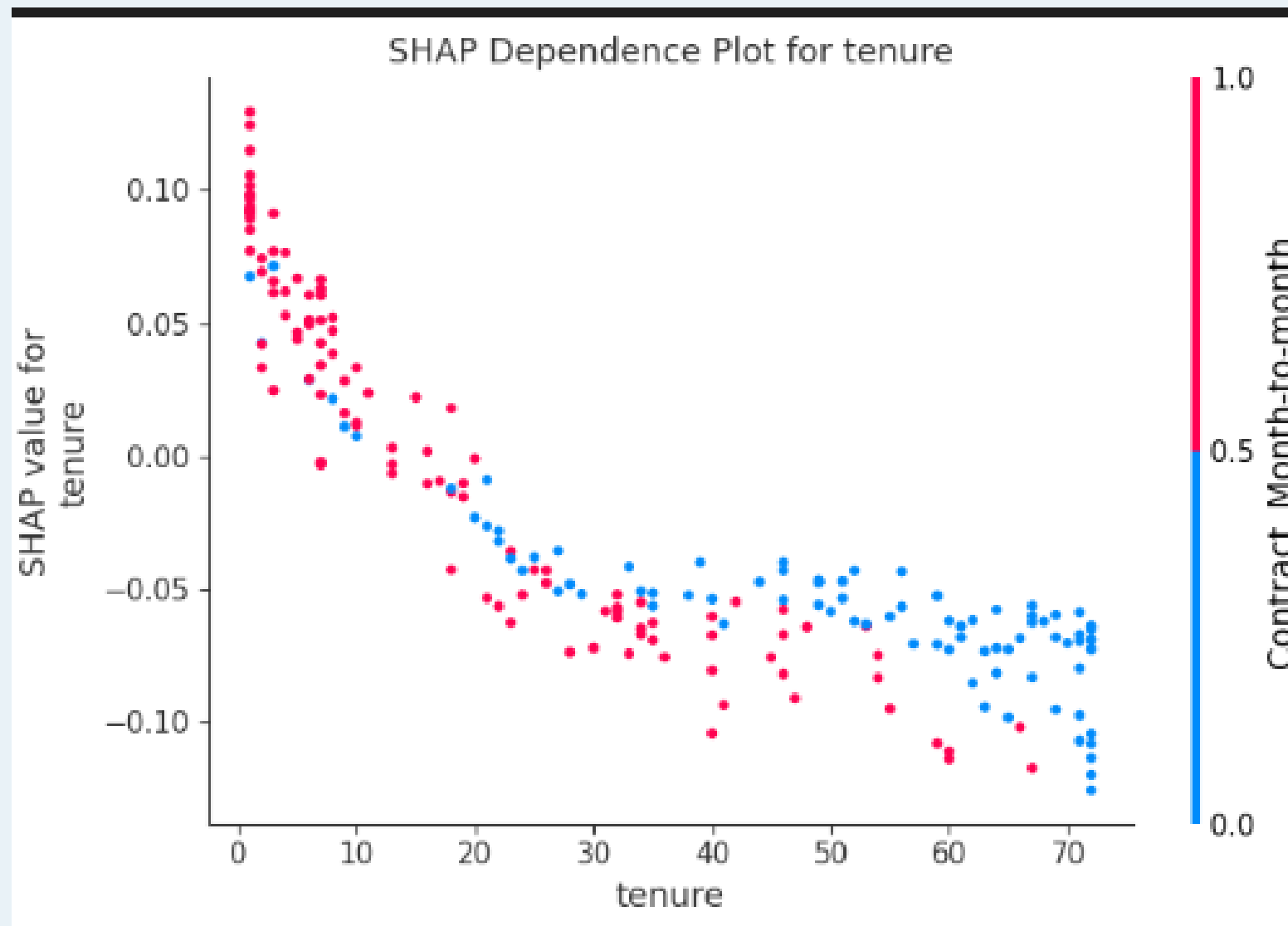
```
shap.summary_plot(  
    shap_values_rf[:, :, 1],  
    X_test_shap_subset,  
    plot_type="dot"  
)
```



SHAP ANALYSIS: Dependence Plots

dependence plots, one per feature:

- Tenure
- Monthly Charges

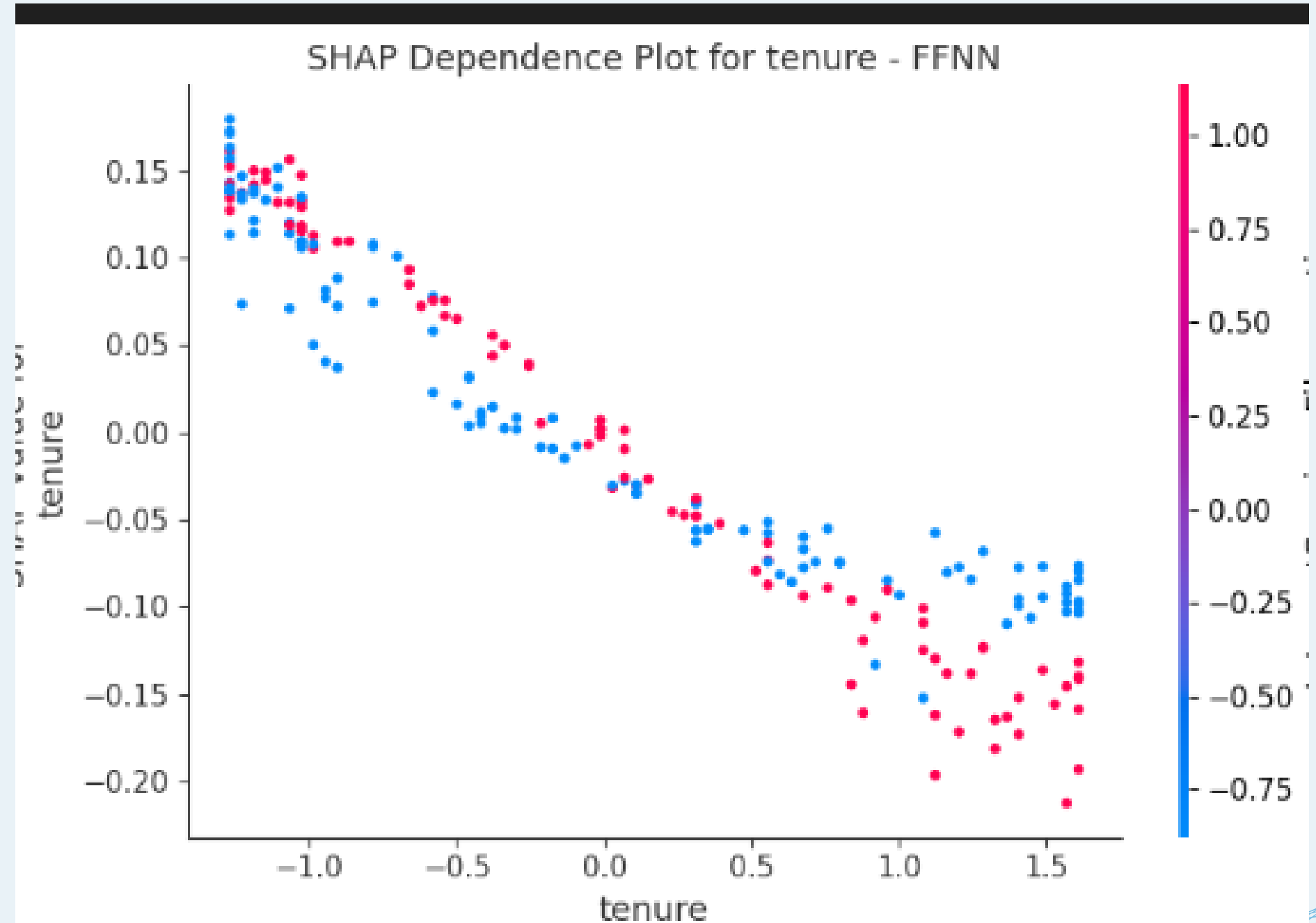


SHAP Dependence Plots – Raw FFNN

Churn Model

Feature impact and interactions on churn prediction :

- Tenure
- MonthlyCharges
- TotalCharges



CONCLUSION:

=>Based on the F1 score, the Soft Voting Ensemble is the best deployment model. It achieves the highest F1-score (0.6264) among all evaluated models.

=>From the interpretability analysis in Part 4, contract and tenure-related features are the strongest contributors to churn. Customers with shorter tenures and less stable contract types are more likely to churn. Monthly charges also play a significant role, with higher charges increasing the risk of churn.

=>Although the ensemble model performs best, there are still opportunities for improvement. Performance could be enhanced by:

- Further hyperparameter tuning of the ensemble components.
- Improved handling of class imbalance, such as adjusting class weights or experimenting with different resampling strategies.
- Feature engineering, especially combining tenure, contract type, and pricing features.

The background features a light blue gradient with abstract decorative elements. In the top-left corner, there are thin, light blue wavy lines. In the top-right corner, there is a dark blue wavy shape with a white geometric pattern of interconnected triangles. In the bottom-left corner, there is a dark blue wavy shape with a white geometric pattern of interconnected triangles. In the bottom-right corner, there are thin, light blue wavy lines.

THANK YOU